Retrieving Complete Bacterial Taxonomic Lineages from NCBI Using Partial Data and Python API

Problem/Aim

In microbiome data analysis, taxonomy assignments—derived from marker genes or whole genomes—are essential for understanding microbial ecosystems when paired with abundance data. These assignments are not limited to single taxon names but are structured as hierarchical lineages across multiple taxonomic ranks, offering richer biological insights. However, researchers often only have partial taxonomy data (e.g., names at a single rank), which limits interpretability and analytical depth.

Solution

To unlock the full potential of taxonomy-based insights, incomplete taxon names can be mapped to their complete hierarchical lineages by retrieving standardized taxonomy data from the NCBI database. This approach enriches the dataset and supports more robust biological interpretation.

Solution Implementation

To retrieve full taxonomy lineages, one can use either the NCBI Taxonomy web browser or a programmatic method. The programmatic approach is highly efficient and advantageous, allowing automation of repetitive retrieval for multiple taxa at once. The NCBI E-utilities Python API enables access to the web server directly from Python scripts, integrating seamlessly with other steps in a microbiome data analysis pipeline.

This section includes two parts:

1. A reusable function to download and parse the full taxonomy lineage for a given taxon at a single rank.
2. An example demonstrating how to apply this function to multiple taxa, using a food fermentation microbiome dataset as a case study.

In [5]:
```
from Bio import Entrez
import pandas as pd
```

Reusable Script

The customizable function below retrieves the full lineage using the Bio.Entrez.esearch(), Bio.Entrez.efetch(), and Bio.Entrez.read() functions from the **Bio.Entrez** submodule in the

Biopython package.

The input is a taxon string at a given rank, and the output is a list of taxa from higher to lower ranks. You can use this function as is or adapt it for your specific needs.

In [ ]:
```python
def get_taxonomy_lineage(rank="Genus", taxon_name=None):
    """Fetches complete taxonomy lineage based on the known taxonomic level (the

    Args:
        rank (str): The taxonomic rank to search for (default is "Genus"). "Famil
        taxon_name (str): The name of the taxon to search for. If None, the funct

    Returns:
        list: A list containing the lineage of the taxon, or None if not found.
    """

    #--- Set the email for NCBI Entrez ---
    # This is required by NCBI to track usage and for contact in case of issues
    # Replace with your email address
    Entrez.email = "davidzhao1015@gmail.com"


    #--- Search for the genus in the NCBI taxonomy database ---
    # The search term is formatted to include the genus name followed by "[Genus]
    # This ensures that the search is limited to the genus level in the taxonomy
    handle = Entrez.esearch(db="taxonomy", term=f"{taxon_name}[{rank}]")
    record = Entrez.read(handle) # Read the search results
    handle.close() # Close the handle to free resources


    #--- Check if any IDs were found for the genus ---
    # If no IDs are found, print a message and return None
    # This is important to handle cases where the genus does not exist in the dat
    if not record["IdList"]:
        print(f"No taxonomy ID found for {rank}: {taxon_name}")
        return None


    #--- Fetch the taxonomy record using the first ID found ---
    # The first ID in the IdList is used to fetch the complete taxonomy record
    # This is because the search may return multiple IDs, but we are interested i
    # The efetch function retrieves the record in XML format for easier parsing
    # The record contains detailed information about the taxonomy, including line
    # The lineage includes domain, kingdom (or clade), phylum, class, order, and
    taxid = record["IdList"][0] # Get the first taxonomy ID from the search resul
    handle = Entrez.efetch(db="taxonomy", id=taxid, retmode="xml")
    records = Entrez.read(handle) # Read the fetched record
    handle.close() # Close the handle to free resources

    #--- Extract the lineage from the fetched record ---
    # The lineage is extracted from the first record in the list of records retur
    # The lineage is a string that includes the complete taxonomy hierarchy for t
    # It is formatted as "domain; kingdom; phylum; class; order; family"
    lineage = records[0]["Lineage"] # Extract the lineage from the record, includ
    if not lineage == None:
        lineage_list = lineage.split("; ") # Split the lineage into a list

    return lineage_list
```

This function is particularly useful for microbiology research where you need to understand the evolutionary relationships and taxonomic classification of bacterial genera commonly found in fermented food and gut microbiome.

In [14]:
```
# Test the function with a specific genus
genus = "Lactobacillus"
lineage = get_taxonomy_lineage(rank="Genus", taxon_name=genus)

print (f"Taxonomy lineage for {genus}: {lineage}")
```

```
Taxonomy lineage for Lactobacillus: ['cellular organisms', 'Bacteria', 'Bacillat
i', 'Bacillota', 'Bacilli', 'Lactobacillales', 'Lactobacillaceae']
```

Example

The example demonstrates applying the function to multiple bacterial taxa using loop iteration in Python, maximizing the efficiency of the Python API. You can adapt the code by replacing it with your own input data.

In [3]:
```
# Define a list of target genera: Fermenting bacteria
genera = ['Acetobacter', 'Gluconacetobacter', 'Lentibacillus', 'Brevibacterium',
          'Kosakonia', 'Lactobacillus', 'Companilactobacillus', 'Schleiferilactob
          'Lactiplantibacillus', 'Loigolactobacillus', 'Paucilactobacillus', 'Lim
          'Acetilactobacillus', 'Secundilactobacillus', 'Lentilactobacillus', 'Ca
          'Enterococcus', 'Tetragenococcus', 'Streptococcus', 'Lactococcus', 'Pec
          'Marinilactobacillus', 'Alkalibacterium', 'Eggerthella', 'Propionibacte

len(genera)
```

Out[3]:
```
36
```

In [6]:
```
#--- Loop through each genus and get the taxonomy lineage ---

# Initialize an empty DataFrame to store all lineages
df_all_lineages = pd.DataFrame()

for genus in genera:
    lineage = get_taxonomy_lineage(genus)
    print(f"Processing genus: {genus}")
    # Create a DataFrame for the current genus
    if lineage is None:
        continue
    if lineage[1] == "Bacteria":
        df_genus_lineage = pd.DataFrame([lineage[1:]])
        df_genus_lineage.columns = ['Domain', 'Kingdom', 'Phylum', 'Class', 'Orde
        df_genus_lineage['Genus'] = genus

        # Append to the main DataFrame
        df_all_lineages = pd.concat([df_all_lineages, df_genus_lineage], ignore_i

genus_successful = df_all_lineages["Genus"].to_list() # Get the list of successfu
print(f"{len(genus_successful)} out of {len(genera)} genera was successful retrie
```

```
Processing genus: Acetobacter
Processing genus: Gluconacetobacter
Processing genus: Lentibacillus
Processing genus: Brevibacterium
Processing genus: Erwinia
Processing genus: Enterobacter
Processing genus: Pantoea
Processing genus: Kosakonia
Processing genus: Lactobacillus
Processing genus: Companilactobacillus
Processing genus: Schleiferilactobacillus
Processing genus: Ligilactobacillus
Processing genus: Lactiplantibacillus
Processing genus: Loigolactobacillus
Processing genus: Paucilactobacillus
Processing genus: Limosilactobacillus
Processing genus: Fructilactobacillus
Processing genus: Acetilactobacillus
Processing genus: Secundilactobacillus
Processing genus: Lentilactobacillus
Processing genus: Carnobacterium
Processing genus: Weissella
Processing genus: Oenococcus
Processing genus: Enterococcus
Processing genus: Tetragenococcus
Processing genus: Streptococcus
Processing genus: Lactococcus
Processing genus: Pediococcus
Processing genus: Periweissella
Processing genus: Leuconostoc
Processing genus: Marinilactobacillus
Processing genus: Alkalibacterium
Processing genus: Eggerthella
Processing genus: Propionibacterium
Processing genus: Staphylococcus
Processing genus: Kocuria
36 out of 36 genera was successful retrieved.
```

In [7]:
```
df_all_lineages.head()
```

Out[7]:

	Domain	Kingdom	Phylum	Class	Order	
0	Bacteria	Pseudomonadati	Pseudomonadota	Alphaproteobacteria	Acetobacterales	Acetoba
1	Bacteria	Pseudomonadati	Pseudomonadota	Alphaproteobacteria	Acetobacterales	Acetoba
2	Bacteria	Bacillati	Bacillota	Bacilli	Bacillales	B
3	Bacteria	Bacillati	Actinomycetota	Actinomycetes	Micrococcales	Breviba
4	Bacteria	Pseudomonadati	Pseudomonadota	Gammaproteobacteria	Enterobacterales	En